Assignment - 2

Module 14 – Python Collections, functions and Modules

- **Accessing List**

1. Understanding how to create and access elements in a list.

⇨ Working with list is a core programming skill and it's actually pretty straightforward once you see the pattern.

Creating a list :-
A list is a just collection of items and written in a square brackets.
List can hold different types too
Example:-
My_list = [10,20,30,40] – creating a list
My_list = [10,20.5,"str",True] – hold a different types also

Accessing Element in a list :-
Example:-

My_list = [10,20,30,40,50]
Print(My_list[1]) **–** output :- 20

Negative Indexing :-
You can also access elements from the end using negative numbers.
Example:-

My_list = [10,20,30,40]
Print(My_list[-1]) – output :- 40

Slicing List :-
You can get a portion of the list using slicing.
Example:-

My_list = [10,20,30,40,50]
Print(My_list[1:4]) **–** output:- 20,30,40

Changing Element :-

List are mutable, meaning you can change the value

Example:-

```
List = [10,20,30,40]

List[1] = 22

Print(List)  - output :- [10,22,30,40]
```

Adding Element :-

Example :-

```
List = [10,20,30]

List.append(40) #add the element at the last

List.insert(1,33) # insert the index 1

print(List) – output :- [10,33,20,30,40]
```

Remove Element :-

Example :-

```
List = [20,40,60,70]

List.remove(60) # direct value through remove

List.pop() #index through remove last item

List.pop(1) #index through remove position

Print(List) – output :- [20]
```

2. Indexing in lists (positive and negative indexing).

⇨ Indexing is how you access individual elements in a list using their position.

- ✓ **Positive Indexing :-**

  Positive indexing starts from the beginning of the list.

  First element is at index 0.

  The second is at index 1 and so on

Example :-

```
L = [10,20,30,40]
Print(L[1])
```

Output :- 20

- ✓ **Negative Indexing :-**
  Negative indexing start at the end of the list.
  First element is at index -1
  Second last element is at index -2

  Example:-

```
L = [10,20,30,40]
Print(L[-2])
```

Output :- 30

3. Slicing a list: accessing a range of elements.

⇨ List slicing lets you access the range of elements from a list using simple and flexible syntax.

Syntax:-

List = [start : stop : step]

Where,

Start = index where slice begin

Stop = index where slice ends

Steps = how much to jump between elements

Examples :-

⇨ **Basic Slicing :-**

```
nums = [10,20,30,40,50]
Print(nums[1:4])
```

Output :- [20,30,40]

- **Omitting start or stop :-**
  Examples:-

```
print(nums[:3]) # from beginning
print(nums[2:]) # till end
```

output:-
[10,20,30]
[20,30,40,50]

- **Using step :-**
  Examples :-

```
Print(nums[::2])
```

Output:-
[10,30,50]

- **Negative indices :-**
  Examples:-

```
print(nums[-3:-1])
```

output:-
[30,40]

- **Reverse List :-**
  Example :-

```
print(::-1)
```

output :- [50,40,30,20,10]

4. Common list operations: concatenation, repetition, membership.

⇨ There are below list operations :-

➢ **Concatenations :-**

Combines two list into one

Example :-

```
List1 = [10,20,30]
List2 = [60,80,40]

Print(List1 + List2)
```

Output:- [10,20,30,60,80,40]

➢ **Repetition :-**

Repeat elements of the list a given number of times.

Example :-

```
list1 = [10,20,30]

print(list1 * 3)
```

Output :- [10,20,30,10,20,30,10,20,30]

➢ **Membership :-**

Check whether an element exist in a list

Example :-

```
list1 = [10,20,30]

print(30 not in list1)
```

output :- False

5. Understanding list methods like append(), insert(), remove(), pop().

⇨ below the brief explanation of above methods

- append() :- adds an element at the end of the list
- insert() :- adds an element at the specific index position.
- Remove() :- remove the first occurrence value.
- Pop():- remove the element by index

Examples :-

```
list1 = [55,67,78,34,89]


print(list1.append(56))

list1.insert(2,44)

list1.remove(34)

list1.pop(1)

print(list1)
```

output :- [55,44,78,89,56]

6. Iterating over a list using loops.

⇨ Iterating over a list simply means going through each element one by one and performing operations.

Example :-

using a For loop

```
my_list = [1,2,3]

for I in my_list:

        print(I)
```

using a index :-

```
my_list = [1,2,3]
for I in range(len(my_list)):
        print(my_list[I])
```

While Loop :-

```
i=0
while i < len(my_list):
        print(my_list[i])
        i+=1
```

7. Sorting and reversing a list using sort(), sorted(), and reverse().

⇨ Below the brief explanations and examples

- **Sort() :-**

  Sort the result.
  Change the original list only.
  Work only on list.

  Example :-

  ```
  my_list = [3, 1, 2]
  my_list.sort()
  print(my_list)
  ```

  output :- [1,2,3]

- **Sorted() :-**

  Create a new list.
  Does not change the original list.
  Works on any iterable (list,tuple,string)

  Example :-

  ```
  my_list = [3, 1, 2]
  new_list = sorted(my_list)
  print(new_list)
  ```

output :- [1,2,3]

- **Reverse() :-**
  Reverse the order of the list

  Example :-
  my_list = [3, 1, 2]
  my_list.reverse()
  print(my_list)

8. Basic list manipulations: addition, deletion, updating, and slicing.

⇨ There are below list manipulations :-

- **Addition :-**
  My_list = [10,20,30,40]
  My_list.append(50)
  My_list.insert(1,33)

  Output :-
  [10,20,30,40,50]
  [10,33,20,30,40]

- **Deletion :-**
  My_list = [10,20,30,40]
  My_list.remove(20)
  My_list.pop(2)

  Output :-
  [10,30,40]
  [10,20,40]

- **Update :-**
  My_list = [10,20,30,40]
  My_list[2] = 60
  Print(My_list)

Output :- [10,20,60,40]

- **Slicing :-**

  My_list = [10,20,30,40]
  Print(My_list[1:4])

  Output :- [20,30,40]

9. Introduction to tuples, immutability.

- Tuple is an built in data-type in python used to store a collection of elements.
- Tuples are immutable.
- Creating a tuple using parenthesis
- You can access the tuple using index.

Example :-

My_list = (1,2,3,4)
Print(my_list[1])

Output :- 2

10. Creating and accessing elements in a tuple.

- A tuple is created using () parenthesis symbol.

Creating a Tuple :-

my_tuple(10,20,30,40)
Print(my_tuple)

Output :- [10,20,30,40]

Accessing a Tuple element :-

my_tuple(10,20,30,40)
Print(my_tuple[2])

Output :- [30]

11. Basic operations with tuples: concatenation, repetition, membership.

⇨ Below the operations with tuples :-

- **Concatenation (+) :-**
  Joining two or more tuples into one

  Example :-

  ```
  s1 = (10,40,60,80)
  s2 = (20,50,30,70)

  print(s1 + s2)
  ```

  output :- (10,40,60,80,20,50,30,70)

- **Repetition (*) :-**
  Repeating element of tuple in multiple times.

  Example :-

  ```
  s1 = (10,40,60,80)
  print(s1*2)
  ```

  output :- (10,40,60,80, 10,40,60,80)

- **Membership Operations (in, not in) :-**
  It is used to check if an element exist in a tuple.

  Example :-

  ```
  s1 = (10,40,60,80)
  print(40 in s1)
  ```

  output :- True

12. Accessing tuple elements using positive and negative indexing.

⇨ In python , tuple supports indexing, which allows you to access elements by their positions.

Positive Indexing :-

```
my_tuple = (10,20,40,50,70)
print(my_tuple[0])
print(my_tuple[1])
print(my_tuple[4])
```

Output :-

```
[10]
[20]
[70]
```

Negative Indexing :-

```
my_tuple = (10,20,40,50,70)
print(my_tuple[-2])
print(my_tuple[-1])
print(my_tuple[-3])
```

Output :-

```
[50]
[70]
[40]
```

13. Slicing a tuple to access ranges of elements.

⇨ Slicing in python allows you to access the range of elements from a tuple using syntax:-
Syntax :-

Tuple[start : stop : step]

Examples :-

1. **Basic Slicing :-**
   my_tuple = (10,20,30,40,50)
   Print(my_tuple[1:4])

   Output :- [20,30,40]

2. **Omitting start or end :-**
   my_tuple = (10,20,30,40,50)
   Print(my_tuple[3:])

   Output :- [40,50]

3. **Using step value :-**
   my_tuple = (10,20,30,40,50)
   Print(my_tuple[0:3:1])

   Output :- [10,30]

4. **Negative slicing**
   my_tuple = (10,20,30,40,50)
   print(my_tuple[-3:])

   output :- [30,40,50]

14. Introduction to dictionaries: key-value pairs.

⇨ Dictionary is a built in data structure used to store the data in key-value pairs.

- A dictionary is ordered and mutable collection of items, where
- Each key is unique
- Each keys map to value

Key Features :-

- Keys must be immutable (e.g like, strings, numbers and tuples)
- Values can be any data type.
- Defined using curly braces {}.

Example :-

```
Student = {
    "name" : "yash",
    "email" : "xyz",
    "age" : 25
}
```

15. Accessing, adding, updating, and deleting dictionary elements.

⇨ Below the brief explanations :-

- **Accessing Element :-**

  you can access the value using their keys.

  Print(student['name'])

  Output :- yash

  Using .get() method :-

  Print(student.get("age"))

  Output :- 25

- **Adding or Update Elements :-**

  You can add the item

  Student["age"] = 21 #update
  Student["city"] = "surat" #add-key value pairs

- **Deleting Element :-**

  You can delete the data using pop() and del()

  Student.pop("course")

  Del student ["age"]

16. Dictionary methods like keys(), values(), and items().

⇨ Below the explanation :-

- **Keys () :-** return all the keys

  Example :-

  ```
  student = {
  "name": "Ravi",
  "age": 20,
  "course": "Python"
  }

  print(student.keys())
  ```

- **Values() :-** return the all the values stored in the dictionary.

  Example :-

  ```
  print(student.values())
  ```

- **Items() :-** return the key-value pair and store the data in dictionary.

  Example :-

  ```
  print(student.items())
  ```

17. Iterating over a dictionary using loops.

⇨ I want to a keys, values and key + values

- **Keys :-**

  Example :-

  ```
  for key in my_dict:
      print(key)
  ```

- **Values :-**

  Example :-

  ```
  for value in my_dict.values():
  ```

```
    print(values)
```

- **Key + value :-**
  Example :-

```
for key, value in my_dict.items():
    print(key,value)
```

18. Merging two lists into a dictionary using loops or zip().

- Using Zip() method :-
  Merging the data and get the what type of data to Insert like (list,tuple etc..)

  Example :-

```
Keys = [“a”,”b”,”c”]

Values = [1,2,3]

My_dict = dict(zip(keys,values))

Print(My_dict)
```

Output :- {‘a’ : 1, ‘b’ : 2, ‘c’ : 3}

19. Counting occurrences of characters in a string using dictionaries.

- Dictonary store the data as key – value pairs.
- Each character in string becomes a key.
- It’s count a becomes the value.

Example :-

```
text = "hello"


count = {}

for char in text:
```

```
    count[char] = count.get(char, 0) + 1


print(count)
```

20. Defining functions in Python.

⇨ A function is reusable block of organize code that perform specific particular task.

Creating a function using def keyword

Syntax :-

```
def function_name(parameter):
    #block of code
    Return value
```

Example :-

```
def message():
    print(“Hello World”)


message()
```

Output :- Hello World

21. Different types of functions: with/without parameters, with/without return values.

⇨ Below the types of functions

✓ **Functions without parameters, without return value :-**

No input, no output

Example :-

```
def greet():
    print("Hello!")

greet()
```

- ✓ **Functions with parameters and without return value :-**
  Take input, but does not return anything
  Example :-
  ```
  def greet(name):
     print("Hello", name)

  greet("Amit")
  ```

- ✓ **Function without parameters and with return value :-**
  No input, but return output
  Example :-
  ```
  def get_number():
     return 10

  print(get_number())
  ```

- ✓ **Functions with parameter and with return value :-**
  Take the input and return output
  Example :-
  ```
  def add(a, b):
     return a + b

  print(add(5, 3))
  ```

22. Anonymous functions (lambda functions).

⇨ Lambda function is a function without name and use only one time and some operations.

Syntax :-

lambda arguments : expression

Example :-

```
Lambda a,b : a+b
Print(add(5,3))
```

23. Understanding how generators work in Python.

⇨ Generator in python are a simple way to produce values at one at a time, instead of all at once.

Example :-

```
def count(n):
    for I in range(n):
        yield I
```

How it works :-

Calling the function doesn't run it fully – it return a generator.

Each next() runs it until next yield.

It pauses and resumes from where it let off.

Why use Generator :-

Save memory

Handle large or infinite data.

Cleaner loops.

24. Difference between yield and return.

⇨ Below the difference of yield and return

Yield	**Return**
Pauses Function	Exit Functions
Can resume later	Cannot resume later
Used in generator functions	Used in regular functions
Saves local state & resumes where it left off.	Ends the functions entirely.

Can run multiple times, one at a time, next call continues after yields.	Runs only once, return final values, No continuations, function starts from beginning each time

25. Understanding iterators and creating custom iterators.

⇨ Below the explanation of iterators and creating custom iterators:-

- **Iterators :-**

An iterator is an object that lets you go through items one by one.

It must have:

iter() :- returns the iterator object

next() :- return the next value

Example :-

```
Num = [1,2,3]

Itr = iter(Num)

Print(next(itr)) #1

Print(next(itr)) #2
```

- **Custom Iterators :-**

Example :-

```
def count(n):

   for i in range(n):

      yield i


for num in count(3):

     print(num)
```

Output :-

0 1 2

26. How functional programming works in Python.

⇨ Functional programming in python in a way of writing code where you focus on functions and data transformation, not changing state.

In simple terms :-
Functions are values -> you can pass them around.
Avoid changing data -> create new data instead
Use built-in helpers -> like map, filter, reduce.

Example :-

```
Num = [1,2,3,4]
Squared = list(map(lambda x : x**2,num))
```

Output:- [1,4,9,16]

27. Using map(), reduce(), and filter() functions for processing data.

⇨ Below the explanations

❖ Map() :- applies a functions to every item.
Example :-

```
a = [1,2,3,4,5,6,7]
k = map(lambda i : i*i,a)
print(list(k))
```

Ouput :- [1,4,9,16,25,36,49]

❖ Reduce() :- reduces a list into single value.
Example :-

```
From functools import reduce
l = [10,20,30,40,50,60]
def sum(a,b):
        return a+b

r = reduce(sum,l)
```

```
print(r)
```

output :- 447

- Filter() :- keeps only items that match a condition.

  Example :-

```
l = [1,2,3,4,5,6,7,8,9,10]
def checkodd(l):
        if l % 2!=0:
            return l

k = filter(lambda i : i % 2!=0,l)
print(k)
```

Output :- [1,3,5,7,9]

28. Introduction to Python modules and importing modules.

- A module is simply a file that contain python code that you can reuse in other programs.

  Why use modules:-

  Resue code

  Keep program organized

  Make code easier to read and maintain.

  Importing Modules :-

- Import entire module :-

  Example :-

```
Import math
Print(math.sqrt(5))
```

- import specific function :-

  Example :-

```
from math import sqrt
print(sqrt(16))
```

- import with alias :-
  Example :-
  Import math as m
  Print(m.sqrt(16))

- import everything :-
  Example :-
  from math import *

29. Standard library modules: math, random.

- Below the explanations :-
- **Math Module :-**
- Used for mathematical operations.
  Example :-

  Import math as m
  Print(m.sqrt(25))

  Output :- 5.0

- **Random Module :-**
- Used to generate the random value.
  Example :-

  Import random as r
  Print(random.randint(1,10))

  Output :- 7

30. Creating custom modules.

- Custom module is just your own python file that contain reusable code.

Step 1 :- create a module file

Create a file name like calc.py

```
Def add(a,b):
    Return a+b


Def greet(name):
    Return f”Hello,  {name}”
```

Step 2 :- use the module in another file (app.py)

```
Import calc as c
Print(m.(10,20))
Print(m.greet(“yash”))
```

Output :-

30

Hello, yash